

Bàn về các bài toán đoạn trong thi tin học

Zhou Xiaobo

Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Bàn về các bài toán đoạn trong thi tin học

IOI China candidate team papers / 2008 / Day 1 / Zhou Xiaobo / interval problems.doc

Tóm tắt

Bài viết giới thiệu một số mô hình bài toán đoạn thường gặp: lập lịch chọn nhiều đoạn nhất, lập lịch với nhiều tài nguyên, lập lịch có hạn chót, phủ đoạn tối thiểu, các biến thể có trọng số, và các bài toán giữa đoạn với điểm. Với mỗi mô hình, bài viết nêu thuật toán, ý tưởng chứng minh đúng đắn và cách cài đặt, rồi minh họa bằng các bài từ ACM-ICPC, CEOI, CTSC và các kỳ thi khác.

Từ khóa: mô hình đoạn; chuyển hóa; tham lam; quy hoạch động; tối ưu.

Dẫn nhập

Trong thi tin học, rất nhiều bài cuối cùng có thể chuyển về bài toán trên các đoạn: chọn một số đoạn thỏa điều kiện, phân các đoạn vào một số tài nguyên, hoặc đặt các đoạn theo một thứ tự nào đó. Lớp bài này có nhiều biến thể và không có một công thức duy nhất. Thường ta phải kết hợp tham lam, quy hoạch động, và các cấu trúc dữ liệu để tối ưu.

Bài viết thảo luận sáu nhóm mô hình:

1. lập lịch đoạn cực đại;
2. lập lịch với nhiều tài nguyên;
3. lập lịch đoạn có hạn chót;
4. phủ đoạn tối thiểu;
5. lập lịch và phủ đoạn có trọng số;
6. các bài toán liên quan giữa đoạn và điểm.

Phần ví dụ được chọn từ ACM-ICPC, CEOI, CTSC và một số nguồn khác. Một vài ví dụ có nhiều thuật toán cùng phân tích thời gian chạy.

1 Lập Lịch Đoạn Cực Đại

Mô hình. Trên trục số có n đoạn. Cần chọn nhiều đoạn nhất sao cho các đoạn được chọn đôi một không giao nhau.

Thuật toán. Sắp xếp các đoạn theo tọa độ đầu phải tăng dần. Duyệt theo thứ tự ấy; nếu đoạn hiện tại không giao với đoạn đã chọn cuối cùng thì chọn nó, ngược lại bỏ qua.

Chứng minh. Rõ ràng các đoạn thuật toán chọn không giao nhau. Ta chứng minh số lượng được chọn là lớn nhất.

Gọi r_i là đầu phải của đoạn thứ i mà thuật toán tham lam chọn, và o_i là đầu phải của đoạn thứ i trong một nghiệm tối ưu bất kỳ, các đoạn trong nghiệm tối ưu cũng được xét theo thứ tự từ trái sang phải.

Mệnh đề 1.1. Với mọi i còn xác định, $r_i \leq o_i$.

Chứng minh bằng quy nạp. Với $i = 1$, thuật toán chọn đoạn có đầu phải nhỏ nhất trong tất cả các đoạn nên mệnh đề đúng. Giả sử đúng với $i = k$. Khi chọn đoạn thứ $k + 1$, mọi đoạn trong nghiệm tối ưu đứng sau o_k đều bắt đầu sau o_k , mà $r_k \leq o_k$, nên chúng cũng là ứng viên hợp lệ sau r_k . Thuật toán chọn ứng viên có đầu phải nhỏ nhất, do đó $r_{k+1} \leq o_{k+1}$.

Giả sử thuật toán chọn m đoạn, nghiệm tối ưu chọn M đoạn. Nếu $M > m$, theo mệnh đề trên ta có $r_m \leq o_m$. Khi đó trong nghiệm tối ưu còn một đoạn thứ $m + 1$ bắt đầu sau o_m , nên cũng bắt đầu sau r_m . Thuật toán không thể dừng sau đoạn thứ m , mâu thuẫn. Vậy $M = m$.

Cài đặt. Khi xét một đoạn mới, chỉ cần so với đoạn được chọn cuối cùng. Độ phức tạp là

$$\mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n).$$

Ví dụ 1: Gene Assembly. Bài Latin America – South America 2001 Problem A cho n exon, mỗi exon có vị trí bắt đầu và kết thúc. Hai exon nối được nếu exon sau bắt đầu sau khi exon trước kết thúc. Cần tìm chuỗi exon dài nhất. Xem mỗi exon là một đoạn; bài toán trở thành lập lịch đoạn cực đại, chỉ cần chú ý điều kiện không được chạm nhau ở đầu mút.

2 Lập Lịch Với Nhiều Tài Nguyên

Mô hình. Có n đoạn và vô hạn tài nguyên. Trên cùng một tài nguyên, các đoạn không được giao nhau. Cần gán mọi đoạn vào tài nguyên sao cho số tài nguyên dùng là nhỏ nhất.

Định nghĩa **độ sâu** d của tập đoạn là số đoạn lớn nhất cùng chứa một điểm nào đó.

Mệnh đề 2.1. Số tài nguyên cần dùng ít nhất là d .

Nếu có d đoạn cùng chứa một điểm, chúng phải nằm trên d tài nguyên khác nhau.

Tính độ sâu. Tách mỗi đoạn thành hai sự kiện đầu trái và đầu phải, sắp xếp theo tọa độ rồi quét từ trái sang phải. Biến đếm tăng khi gặp đầu trái và giảm khi gặp đầu phải; giá trị lớn nhất của biến đếm là d . Khi hai sự kiện cùng tọa độ, thứ tự xử lý phụ thuộc vào việc các đoạn được phép chạm đầu mút hay không. Độ phức tạp là $\mathcal{O}(n \log n)$.

2.1 Thuật toán theo đầu trái

Tính d . Sắp xếp các đoạn theo đầu trái tăng dần. Khi xử lý một đoạn, loại các tài nguyên đang được các đoạn trước đó giao với nó sử dụng, rồi chọn một tài nguyên còn lại trong $1, 2, \dots, d$.

Mệnh đề 2.2. Mỗi đoạn đều gán được vào một tài nguyên.

Xét đoạn I . Nếu có k đoạn trước nó giao với I , thì tất cả các đoạn ấy chứa đầu trái của I . Do định nghĩa độ sâu, $k + 1 \leq d$, nên trong d tài nguyên còn ít nhất một tài nguyên chưa bị loại.

Mệnh đề 2.3. Hai đoạn giao nhau không bao giờ được gán cùng tài nguyên.

Nếu I đứng trước J và hai đoạn giao nhau, khi xử lý J , tài nguyên của I đã bị loại. Vậy thuật toán tạo được nghiệm dùng đúng d tài nguyên; theo cận dưới ở mệnh đề 2.1, đó là tối ưu.

Trong cài đặt, có thể không cần tính d trước. Duy trì đầu phải lớn nhất hiện tại của mỗi tài nguyên bằng hàng đợi ưu tiên; nếu tài nguyên có đầu phải nhỏ nhất không dùng được, mở tài nguyên mới. Tổng thời gian vẫn là $\mathcal{O}(n \log n)$.

Mệnh đề 2.4. Số tài nguyên tối thiểu bằng độ sâu d của tập đoạn.

2.2 Thuật toán theo đầu phải

Từ mệnh đề 2.4, ta có một cách tham lam khác. Tính d , sắp xếp các đoạn theo đầu phải tăng dần. Với mỗi đoạn, gán nó vào một tài nguyên khả dụng có đầu phải hiện tại lớn nhất. “Khả dụng” nghĩa là đoạn cuối trên tài nguyên ấy không giao với đoạn đang xét.

Chứng minh dùng so sánh trạng thái. Sau khi xử lý i đoạn, ghi lại trong một bảng có thứ tự A các đầu phải cuối cùng của d tài nguyên trong thuật toán; với một nghiệm tối ưu tương ứng, ghi bảng B . Ta nói B không tốt hơn A nếu từng phần tử của A không lớn hơn phần tử tương ứng của B . Quy nạp cho thấy sau mỗi bước tồn tại một nghiệm tối ưu có trạng thái không tốt hơn trạng thái thuật toán. Vì thuật toán luôn chọn tài nguyên khả dụng có đầu phải lớn nhất, trạng thái mới của thuật toán vẫn không tệ hơn sau khi chèn đầu phải của đoạn hiện tại. Do nghiệm tối ưu luôn gán được đoạn tiếp theo, thuật toán cũng gán được.

Cài đặt thuận tiện nhất là cây cân bằng lưu các đầu phải hiện tại. Mỗi đoạn cần tìm đầu phải lớn nhất nhỏ hơn đầu trái của nó rồi cập nhật; tổng thời gian $\mathcal{O}(n \log n)$.

Thuật toán đầu tiên chứng minh trực tiếp số tài nguyên tối thiểu và dễ cài đặt. Thuật toán thứ hai dựa trên tính tối ưu cục bộ mạnh hơn, nên thuận tiện khi bài toán có thêm điều kiện phụ.

Ví dụ 2: Berth Allocation. Bài Guangdong Provincial Collegiate Programming Contest 2004 Problem B mô tả một cảng chia thành nhiều khu, mỗi khu có sức chứa cố định. Mỗi tàu có thời điểm đến, rời cảng và khu bắt buộc. Cần nhận nhiều tàu nhất.

Mỗi khu được xử lý độc lập. Xem mỗi tàu là một đoạn thời gian, số tài nguyên là sức chứa của khu. Sắp xếp theo đầu phải; nếu còn tài nguyên khả dụng thì xếp tàu vào tài nguyên khả dụng có đầu phải lớn nhất, nếu không thì từ chối tàu. Lập luận đổi chỗ chuẩn cho thấy khi thuật toán nhận một tàu mà một nghiệm tối ưu không nhận, có thể thay một tàu giao với nó và kết thúc không sớm hơn bằng tàu hiện tại mà không giảm số tàu. Dùng cây cân bằng, thời gian cho mỗi khu là $\mathcal{O}(n \log n)$.

3 Lập Lịch Đoạn Có Hạn Chót

Mô hình. Có n đoạn có độ dài cố định nhưng vị trí có thể thay đổi. Đoạn i có độ dài p_i , hạn chót d_i , và đầu phải khi đặt là r_i . Độ trễ của nó là $L_i = r_i - d_i$. Cần đặt tất cả đoạn không giao nhau sao cho

$$L_{\max} = \max_i L_i$$

nhỏ nhất.

Thuật toán. Sắp xếp các đoạn theo hạn chót tăng dần, rồi đặt liên tiếp không để khoảng trống.

Chứng minh. Mệnh đề 3.1. Tồn tại một nghiệm tối ưu không có khoảng trống.

Nếu giữa hai đoạn kề nhau còn khoảng trống, dời toàn bộ phần sau sang trái cho tới khi khoảng trống biến mất. Mọi đầu phải ở phần sau không tăng, do đó độ trễ cực đại không tăng. Lặp lại thao tác này sẽ thu được nghiệm tối ưu không có khoảng trống.

Mệnh đề 3.2. Mọi nghiệm không có nghịch thế theo hạn chót và không có khoảng trống có cùng độ trễ cực đại.

Khi thứ tự đã không giảm theo hạn chót, các đoạn có cùng hạn chót chỉ có thể đổi chỗ lẫn nhau; đầu phải của đoạn cuối trong nhóm không đổi, nên giá trị lớn nhất liên quan tới nhóm ấy không đổi.

Mệnh đề 3.3. Một nghiệm không có nghịch thế và không có khoảng trống là tối ưu.

Lấy một nghiệm tối ưu không có khoảng trống. Nếu hai đoạn kề nhau i, j bị nghịch thế, tức $d_i > d_j$ nhưng i đứng trước j , hoán đổi chúng. Đầu phải mới của j không lớn hơn đầu phải cũ của i , còn đầu phải mới của i bằng đầu phải cũ của j ; vì $d_j < d_i$, độ trễ cực đại của cặp không tăng. Liên tục hoán đổi các nghịch thế sẽ thu được thứ tự theo hạn chót mà vẫn tối ưu.

Cài đặt chỉ cần sắp xếp và quét: $\mathcal{O}(n \log n)$.

Ví dụ 3.1: CTSC 2007 Day 2, Pendant. Mỗi hạt trang sức có khối lượng w_i và sức chịu c_i . Một dây treo ổn định nếu với mỗi hạt, tổng khối lượng các hạt bên dưới không vượt quá sức chịu của nó. Cần chọn nhiều hạt nhất, và trong số đó tổng khối lượng nhỏ nhất.

Xem hạt i là đoạn có độ dài w_i , vị trí có thể thay đổi, đầu trái không được vượt quá c_i . Ta chọn nhiều đoạn nhất, đặt liên tiếp từ trái sang phải, và khi số đoạn tối đa đã cố định thì cực tiểu hóa đầu phải cuối cùng. Có thể chứng minh như trên rằng tồn tại nghiệm tối ưu sắp theo c_i .

Một quy hoạch động tự nhiên là sắp theo c_i , đặt $f[j]$ là đầu phải nhỏ nhất sau khi chọn j hạt. Với hạt i , hoặc bỏ qua nó, hoặc nếu $f[j] + w_i$ hợp lệ thì cập nhật $f[j + 1]$. Dạng thô là $\mathcal{O}(n^2)$. Vì $f[j]$ tăng theo j , có thể duy trì dãy bằng cây cân bằng và thực hiện thao tác chèn, dịch đoạn trong $\mathcal{O}(\log n)$, đạt $\mathcal{O}(n \log n)$.

Bản gốc cũng nêu một thuật toán tham lam tốt hơn về ý tưởng: xử lý theo khối lượng tăng dần, duy trì một bảng có thứ tự theo sức chịu; nếu hạt hiện tại có thể chèn vào bảng thì chọn, nếu không thì bỏ. Bảng có thể được duy trì bằng cây đoạn, cũng đạt $\mathcal{O}(n \log n)$, nhưng cài đặt chi tiết khá dài.

Ví dụ 3.2: Loan Scheduling. Có n hồ sơ vay, hồ sơ i có hạn chót d_i và lợi nhuận p_i ; mỗi hồ sơ tốn một đơn vị thời gian. Ngân hàng xử lý tối đa L hồ sơ tại cùng một thời điểm. Cần chọn và đặt các hồ sơ để lợi nhuận lớn nhất.

Đây là bài chọn các đoạn đơn vị có trọng số, đặt trên L tài nguyên, sao cho vị trí không vượt quá hạn chót. Sắp các hồ sơ theo lợi nhuận giảm dần. Với mỗi hồ sơ, nếu còn một vị trí không muộn hơn d_i có số hồ sơ đang đặt nhỏ hơn L , đặt nó vào vị trí lớn nhất như vậy; nếu không thì bỏ.

Có thể dùng cây đoạn duy trì số chỗ còn lại trên mỗi thời điểm, xử lý mỗi hồ sơ trong $\mathcal{O}(\log D)$. Tốt hơn, dùng DSU: khi một thời điểm đã đủ L hồ sơ, gộp nó với thời điểm trước đó. Khi cần đặt hồ sơ hạn chót d_i , tìm đại diện của d_i ; đó là thời điểm muộn nhất còn chỗ. Cách này có chi phí gần tuyến tính sau bước sắp xếp.

4 Phủ Đoạn Tối Thiểu

Mô hình. Có n đoạn. Cần chọn ít đoạn nhất sao cho chúng phủ kín đoạn mục tiêu $[s, t]$.

Thuật toán. Sắp xếp các đoạn theo đầu trái tăng dần. Giả sử đã phủ tới điểm x , ban đầu $x = s$. Trong các đoạn có đầu trái không vượt quá x , chọn đoạn có đầu phải lớn nhất, rồi cập nhật x thành đầu phải ấy. Lặp cho tới khi $x \geq t$.

Chứng minh. Gọi r_i là điểm phủ xa nhất sau khi thuật toán chọn i đoạn, và o_i là điểm phủ xa nhất sau i đoạn đầu của một nghiệm tối ưu.

Mệnh đề 4.1. Với mọi i , $r_i \geq o_i$.

Quy nạp theo i . Ở bước đầu, thuật toán chọn đoạn phủ s vươn xa nhất. Giả sử $r_i \geq o_i$. Đoạn thứ $i + 1$ của nghiệm tối ưu phải bắt đầu không quá o_i , nên cũng không quá r_i , hoặc nếu nó chỉ cần nối sau r_i thì càng không thể vượt lợi thế của thuật toán. Thuật toán chọn đoạn vươn xa nhất trong tất cả đoạn dùng được, nên $r_{i+1} \geq o_{i+1}$.

Nếu thuật toán dùng m đoạn còn nghiệm tối ưu dùng ít hơn m , thì theo mệnh đề trên sau số đoạn của nghiệm tối ưu, thuật toán đã phủ xa ít nhất bằng nghiệm tối ưu, tức đã phủ tới t , mâu thuẫn với việc nó còn cần thêm đoạn. Vậy thuật toán tối ưu.

Cài đặt bằng một lần quét sau khi sắp xếp, tổng thời gian $\mathcal{O}(n \log n)$.

Ví dụ 4: Watering Grass. Bài ACM/ICPC Regional Warm-up Contest 2002 Problem E cho một bãi cỏ dài l , rộng w , các vòi phun nằm trên đường giữa, vòi i ở vị trí x_i với bán kính r_i . Cần bật ít vòi nhất để phủ cả bãi.

Một vòi có ích nếu $r_i > w/2$. Khi đó nó phủ trên trục dài một đoạn

$$\left[x_i - \sqrt{r_i^2 - (w/2)^2}, x_i + \sqrt{r_i^2 - (w/2)^2} \right].$$

Bài toán chuyển đúng thành phủ đoạn $[0, l]$ tối thiểu.

Hình 4.1. Mặt cắt bãi cỏ: vòng tròn tưới của một vòi cắt mép bãi thành một đoạn hữu ích trên chiều dài.

5 Lập Lịch Và Phủ Đoạn Có Trọng Số

Khi đoạn có trọng số, các thuật toán tham lam ở trên không còn đúng trong trường hợp tổng quát. Phương pháp phổ biến hơn là quy hoạch động, sau đó tối ưu bằng cấu trúc dữ liệu hoặc tính đơn điệu.

Ví dụ 5.1: Joke with Turtles. Mỗi rùa khai hai số a_i, b_i : số rùa ở trước và ở sau nó. Vì các rùa có thể đồng hạng, nếu rùa nói thật thì vị trí của nó nằm trong một đoạn xác định bởi a_i, b_i . Nếu nhiều rùa tạo cùng đoạn, số rùa có thể cùng nằm trong đoạn ấy bị chặn bởi độ dài hợp lệ của nhóm đồng hạng. Gán cho mỗi đoạn một trọng số là số rùa tối đa có thể nói thật trên đoạn đó. Cần chọn các đoạn không giao nhau sao cho tổng trọng số lớn nhất.

Sắp các đoạn theo đầu phải. Gọi $p(i)$ là chỉ số đoạn cuối cùng nằm hoàn toàn bên trái đoạn i . Đặt $dp[i]$ là tổng trọng số lớn nhất khi chỉ xét i đoạn đầu, ta có công thức chuẩn:

$$dp[i] = \max\{dp[i - 1], dp[p(i)] + w_i\}.$$

Số rùa nói dối ít nhất là $n - dp[n]$. Với miền tọa độ nhỏ, các bước sắp xếp và tìm $p(i)$ có thể làm tuyến tính bằng radix sort hoặc bucket.

Ví dụ 5.2: Cleaning Shifts. Cần phủ mọi giây từ M đến E . Ứng viên i làm trong đoạn nguyên $[l_i, r_i]$ với chi phí c_i ; nếu thuê thì phải thuê cả đoạn. Cần chi phí nhỏ nhất để phủ kín.

Sắp các đoạn theo đầu phải. Đặt $f[i]$ là chi phí nhỏ nhất của một cách phủ tới r_i và bắt buộc chọn đoạn i cuối cùng. Khi chọn đoạn i , đoạn trước đó phải phủ tới ít nhất $l_i - 1$. Công thức có dạng

$$f[i] = c_i + \min f[j],$$

trong đó j chạy trên các đoạn có thể nối với i . Đáp án là giá trị nhỏ nhất trong các trạng thái phủ được tới E .

Bản gốc nêu ba hướng tối ưu:

- dùng cây đoạn trên trục thời gian, mỗi khi có $f[i]$ thì cập nhật tại r_i , còn khi tính $f[i]$ thì lấy min trên khoảng nối được;
- dùng một ngăn xếp đơn điệu để duy trì các trạng thái hữu ích, rồi tìm bằng nhị phân;
- sắp theo đầu trái và duy trì hàng đợi ưu tiên theo giá trị f ; khi xử lý đoạn hiện tại, loại các trạng thái không còn nối được, dùng trạng thái có f nhỏ nhất để chuyển, rồi đưa đoạn hiện tại vào hàng đợi.

Hướng hàng đợi ưu tiên có thời gian $\mathcal{O}(n \log n)$, hệ số vừa phải, và đặc biệt ngắn nếu dùng C/C++.

6 Các Bài Toán Giữa Đoạn Và Điểm

Mô hình. Có n đoạn và m điểm. Một đoạn và một điểm ghép được nếu đoạn chứa điểm ấy. Cần chọn nhiều cặp ghép nhất, mỗi đoạn và mỗi điểm dùng không quá một lần.

Dựng đồ thị hai phía rồi chạy matching là quá chậm; ta phải tận dụng cấu trúc đoạn.

Thuật toán. Sắp các điểm theo tọa độ tăng dần. Khi xét một điểm x , trong các đoạn còn lại chứa x , chọn đoạn có đầu phải nhỏ nhất để ghép với x . Nếu không có đoạn nào chứa x , bỏ qua điểm này.

Chứng minh. Gọi cặp đầu tiên thuật toán chọn là (x, I) , trong đó I là đoạn chứa x có đầu phải nhỏ nhất.

Mệnh đề 6.1. Trong một nghiệm tối ưu bất kỳ, ít nhất một trong x hoặc I được dùng.

Nếu cả hai đều không dùng, thêm cặp (x, I) sẽ tăng số cặp, mâu thuẫn tối ưu.

Mệnh đề 6.2. Tồn tại một nghiệm tối ưu chứa mọi cặp mà thuật toán chọn.

Xét cặp (x, I) . Nếu nghiệm tối ưu đã chứa nó thì xong. Nếu chỉ chứa x , giả sử x đang ghép với J , thay (x, J) bằng (x, I) . Nếu chỉ chứa I , giả sử I đang ghép với y , thay (y, I) bằng (x, I) . Nếu cả x và I đều được dùng nhưng ghép với đối tượng khác, giả sử (x, J) và (y, I) . Vì x là điểm đang xét sớm nhất và I có đầu phải nhỏ nhất trong các đoạn chứa x , có thể đổi thành (x, I) và (y, J) mà vẫn hợp lệ. Số cặp không đổi. Lặp phép đổi này sẽ thu được nghiệm tối ưu chứa các lựa chọn của thuật toán.

Cài đặt hiệu quả: sắp các đoạn theo đầu trái, sắp các điểm theo tọa độ. Duyệt điểm từ trái sang phải; đưa vào hàng đợi ưu tiên mọi đoạn có đầu trái không vượt quá điểm hiện tại, loại khỏi hàng đợi các đoạn có đầu phải nhỏ hơn điểm hiện tại, rồi lấy đoạn có đầu phải nhỏ nhất để ghép. Mỗi đoạn vào và ra hàng đợi một lần, tổng thời gian

$$\mathcal{O}(n \log n + m \log m + n \log n).$$

Ví dụ 6.1: CEOI 2004 Trips. Có n nhóm khách và m chuyến đi. Nhóm i có kích thước s_i ; chuyến j nhận nhóm có kích thước trong $[l_j, u_j]$. Mỗi nhóm và mỗi chuyến dùng tối đa một lần. Xem mỗi chuyến là một đoạn $[l_j, u_j]$, mỗi nhóm là một điểm s_i . Bài toán trở thành ghép nhiều điểm-đoạn nhất.

Ví dụ 6.2: Enlightened Landscape. Có một đường núi gấp khúc và N bóng đèn cùng độ cao. Một điểm trên núi được chiếu sáng bởi bóng đèn nếu đoạn nối giữa chúng không bị núi chắn. Cần bật ít đèn nhất để chiếu sáng toàn bộ cảnh quan.

Điều kiện cần và đủ là mọi đỉnh gấp khúc đều được chiếu sáng. Với một đỉnh cố định, các bóng đèn chiếu được nó tạo thành một đoạn liên tiếp trong dãy bóng đèn. Vì vậy bài toán chuyển thành: cho một số đoạn trên tập các điểm rời rạc (các bóng đèn), chọn ít điểm nhất sao cho mỗi đoạn chứa ít nhất một điểm được chọn.

Trước hết xóa các đoạn chứa đoạn khác, vì chúng không ràng buộc hơn. Có thể sắp theo đầu phải tăng dần, nếu trùng thì theo đầu trái giảm dần, rồi quét và giữ đầu trái lớn nhất đã thấy. Sau đó xét các đoạn còn lại theo thứ tự; nếu đoạn đã chứa một điểm được chọn thì bỏ qua, ngược lại chọn đầu phải của nó. Đây là đối ngẫu rời rạc của phủ đoạn tham lam.

Hình 6.1. Với một đỉnh của đường núi, các bóng đèn nhìn thấy được tạo thành một đoạn liên tiếp trong thứ tự từ trái sang phải của các bóng đèn.

Nếu tính trực tiếp đỉnh nào nhìn thấy bóng nào, thời gian là $\mathcal{O}(MN)$, đủ cho ràng buộc nhỏ của đề gốc. Bản gốc cũng phác thảo cách dùng ngăn xếp để tìm biên trái và biên phải của đoạn bóng đèn nhìn thấy được cho mỗi đỉnh. Mỗi đỉnh vào và ra ngăn xếp nhiều nhất một lần; kết hợp tìm kiếm nhị phân trên dãy bóng đèn, tiền xử lý giảm còn $\mathcal{O}(M \log N)$.

Kết luận

Bài toán đoạn thường xoay quanh việc giữ một thứ tự hợp lý. Chọn đúng khóa sắp xếp là bước rất quan trọng: đầu trái, đầu phải, hạn chót, trọng số hay một giá trị trạng thái đều có thể là chìa khóa của lời giải.

Hai phương pháp chính là tham lam và quy hoạch động. Quy hoạch động có phạm vi áp dụng rộng hơn, nhưng tham lam nếu đúng thường nhanh hơn, dễ cài đặt hơn và dễ tối ưu hơn.

Nhiều tối ưu bài toán đoạn có thể dùng cây đoạn, và một số ít cần cây cân bằng. Tuy nhiên đó không nhất thiết là cách tốt nhất. Khi phân tích sâu hơn, ta thường tìm được tính đơn điệu của quyết định hoặc một cấu trúc dữ liệu đơn giản hơn như hàng đợi ưu tiên, ngăn xếp, hoặc DSU; những cách này vừa ngắn gọn vừa có hệ số thời gian tốt hơn.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
2. Wang Xiaodong, *Thiết kế và phân tích thuật toán máy tính*.
3. Jon Kleinberg, Eva Tardos, *Algorithm Design*.
4. *Đề thi Guangdong Collegiate Programming Contest, 2003–2005*.
5. CTSC 2007 Reports, Guo Huayang.
6. Wang Ting, báo cáo lời giải bài *Turtle*, bài tập đội tuyển năm 2005.
7. Zhu Chenguang, *Ứng dụng các cấu trúc dữ liệu cơ bản trong thi tin học*, luận văn đội tuyển năm 2006.

A Ghi chú về các đề trong phụ lục gốc

Phụ lục của bản gốc chép lại đầy đủ nhiều đề tiếng Anh. Bản dịch này giữ vai trò của phụ lục bằng cách nêu gọn các đề đã được dùng trong phần chính:

Gene Assembly

Chọn chuỗi exon dài nhất theo thứ tự xuất hiện, tương ứng với lập lịch đoạn cực đại.

Who is Still Alive?

Tìm các khoảng thời gian có nhiều tổng thống còn sống nhất; đây là ví dụ trực tiếp của tính độ sâu tập đoạn.

Berth Allocation

Chọn nhiều tàu nhất để xếp vào các khu cảng có sức chứa theo thời gian, tương ứng với lập lịch nhiều tài nguyên có giới hạn.

Pendant

Chọn dây hạt ổn định dài nhất, rồi cực tiểu hóa tổng khối lượng; bài được chuyển thành chọn và đặt các đoạn có hạn chót.

Loan Scheduling

Chọn các khoản vay đơn vị thời gian với lợi nhuận và hạn chót, xử lý trên L tài nguyên song song.

Watering Grass

Chuyển mỗi vòi phun thành một đoạn phủ hữu ích trên chiều dài bãi cỏ, rồi dùng phủ đoạn tối thiểu.

Joke with Turtles

Chuyển mỗi lời khai đúng thành một đoạn vị trí có trọng số, rồi chọn các đoạn không giao nhau có tổng trọng số lớn nhất.

Cleaning Shifts

Chọn các đoạn có chi phí để phủ mọi điểm thời gian trong ca làm việc với tổng chi phí nhỏ nhất.

Trips Ghép kích thước nhóm khách với khoảng kích thước cho phép của chuyến đi, tức ghép điểm với đoạn.

Enlightened Landscape

Với mỗi đỉnh cảnh quan, các bóng đèn nhìn thấy được tạo thành một đoạn; cần chọn ít bóng đèn nhất đâm qua mọi đoạn.